

Findings and Remediation Report

Enterprise DevSecOps Security Lab

Author: Jagriti Banerjee

AppSec	Docker	Kubernetes	GitOps	Runtime Security	Supply Chain
--------	--------	------------	--------	------------------	--------------

Report Scope	This document summarizes security findings discovered and remediated in a controlled private DevSecOps lab. It is written in a professional assessment format to demonstrate vulnerability analysis, attack-path thinking, remediation planning, validation evidence, and security engineering maturity.
--------------	--

Vulnerable App	Security Testing	Finding Analysis	Remediation	Validation	Evidence
----------------	------------------	------------------	-------------	------------	----------

Area	Coverage
Application Security	SQL injection, command injection, dependency scanning, SAST findings
Container Security	Docker hardening, non-root execution, image scanning, SBOM and signing
Kubernetes Security	SecurityContext, RBAC, NetworkPolicy, Pod Security, admission control
Runtime Security	Falco detection, privileged pod detection, sensitive file access alerting
GitOps and Observability	ArgoCD drift correction, Prometheus metrics, Grafana dashboards

1. Executive Summary

This findings and remediation report documents the security issues intentionally introduced, detected, validated, and remediated in the Enterprise DevSecOps Security Lab. The project simulates a modern cloud-native software delivery environment using a vulnerable Flask application, Docker, GitHub Actions, Kind Kubernetes, ArgoCD, Cosign, Syft, OPA Gatekeeper, Falco, Prometheus, and Grafana.

The assessment followed a practical security-engineering workflow: identify vulnerable behavior, prove exploitability in a safe private environment, map impact, implement controls, and validate that the remediation either blocks the attack path or improves detection and visibility.

Overall Outcome		The lab moved from intentionally vulnerable application and Kubernetes states to a hardened DevSecOps posture with secure container execution, least-privilege access, network segmentation, supply-chain evidence, admission-control rules, GitOps drift control, runtime detection, and monitoring dashboards.
Severity	Count	Primary Risk Themes
Critical	4	Container escape pattern, command execution risk, insecure pod context, cluster-admin RBAC
High	5	SQL injection, dependency exposure, image hardening gap, network exposure, unsigned artifact risk
Medium	3	GitOps drift, admission metadata control, visibility/observability gaps
Low	0	No low-severity findings were prioritized for this report

The highest-risk themes were Kubernetes privilege boundaries, overprivileged service accounts, unsafe command execution, and missing preventive controls before policy enforcement. These are common real-world failure modes in cloud-native environments and therefore provide strong portfolio value when documented with evidence and remediation.

2. Scope and Assessment Method

The assessment was limited to a private VM-based lab environment. No third-party or public systems were tested. The scope included source code, container image configuration, CI/CD security checks, Kubernetes manifests, GitOps deployment behavior, supply-chain artifacts, runtime detection, and monitoring.

In Scope	Out of Scope
Flask application routes and dependencies	Public internet targets or real organizations
Dockerfile, local image, GHCR image, Trivy scans	Unauthorized exploitation or external vulnerability testing
Kubernetes Deployment, RBAC, NetworkPolicy, Pod Security	Production clusters, real customer data, or cloud accounts
ArgoCD, Cosign, Syft, Gatekeeper, Falco, Prometheus, Grafana	Red-team actions outside the private lab

Risk ratings are assigned using a practical CVSS-style approach. The score reflects exploitability, blast radius, required privileges, impact to confidentiality/integrity/availability, and whether the issue could lead to code execution, secret access, or cluster compromise in a realistic environment.

Severity	Definition Used in This Report
Critical	Direct path to container escape, cluster compromise, privileged execution, or unsafe command execution with major impact.
High	Significant data exposure, unauthorized access, vulnerable dependency exposure, or control weakness with realistic exploitability.
Medium	Missing preventive, detective, or governance control that increases operational or audit risk.
Low	Hardening improvement with limited direct exploitability.

3. Findings Summary

ID	Finding	Severity	Score	Status
FND-001	SQL injection in Flask /user endpoint	High	8.1	Documented
FND-002	Command injection in Flask /ping endpoint	Critical	9.0	Documented
FND-003	Vulnerable Python dependency exposure	High	7.5	Mitigated by pipeline control
FND-004	Container image vulnerability and hardening gap	High	7.2	Remediated
FND-005	Insecure Kubernetes pod security context	Critical	9.1	Remediated
FND-006	Privileged pod with hostPath container escape pattern	Critical	9.3	Remediated
FND-007	Overprivileged ServiceAccount with cluster-admin access	Critical	9.0	Remediated
FND-008	Missing default-deny network segmentation	High	7.4	Remediated
FND-009	Kubernetes configuration drift without GitOps control	Medium	6.0	Remediated
FND-010	Unsigned image and missing SBOM/provenance evidence	High	7.2	Remediated
FND-011	Missing admission-control policy for supply-chain metadata	Medium	6.4	Remediated
FND-012	Runtime security and observability visibility gaps	Medium	6.2	Remediated

Remediation Theme

The remediation strategy combined secure coding, dependency hygiene, container hardening, Kubernetes least privilege, network isolation, policy-as-code, supply-chain attestations, runtime detection, and observability. This is the same layered model expected in modern DevSecOps programs.

Prevent	Harden	Detect	Validate	Document
---------	--------	--------	----------	----------

4. Detailed Findings and Remediation

FND-001	SQL injection in Flask /user endpoint	High	8.1	Documented
Affected Component	Flask route /user			
Attack Class	Injection - unsafe SQL query construction			
Evidence	SQL injection payload returned more data than expected			
Business Impact	Unauthorized data access and potential database manipulation if expanded beyond a lab demo			

Description

The application used direct string interpolation to construct a SQL query using the user-controlled id parameter. This allowed the input to alter the query logic and return records outside the intended lookup.

```
query = f"SELECT id, username, role FROM users WHERE id = '{user_id}'"
# Example payload: /user?id=1' OR '1'='1
```

Impact

- Bypass of intended record lookup logic.
- Unauthorized enumeration of database records.
- Potential data modification or deletion if write queries are added later.
- Reputational and compliance risk in a real application.

Remediation

- Replace string interpolation with parameterized queries.
- Validate that the id parameter is numeric before database access.
- Use centralized database access helpers to prevent repeat mistakes.
- Add SAST and unit tests that fail when unsafe query patterns are introduced.

```
cursor.execute(
    "SELECT id, username, role FROM users WHERE id = ?",
    (user_id,)
)
```

Validation

After remediation, the same payload should be treated as data rather than executable query logic. The endpoint should return either a single valid record or a controlled validation error.

```
jazz@ubuntu-endpoint: ~/devsecops-k8s-security-lab$
Normal user lookup:
{"query_used": "SELECT id, username, role FROM users WHERE id = '1'", "result": [[1, "alice", "admin"]]}

SQL Injection test:
{"query_used": "SELECT id, username, role FROM users WHERE id = '1' OR '1'='1'", "result": [[1, "alice", "admin"], [2, "bob", "developer"], [3, "charlie", "intern"]]}

(.venv) jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$
```

Evidence: SQL injection proof in the private Flask lab.

FND-002	Command injection in Flask /ping endpoint	Critical	9.0	Documented
Affected Component	Flask route /ping			
Attack Class	Command injection through shell execution			
Evidence	Injected command executed through the host parameter			
Business Impact	Potential arbitrary command execution inside the container			

Description

The ping endpoint passed user-controlled input into subprocess.run with shell=True. This allowed command separators to append additional shell commands to the intended ping command.

```
result = subprocess.run(
    f"ping -c 1 {host}",
    shell=True,
    capture_output=True,
    text=True
)
```

Impact

- Arbitrary OS command execution inside the container context.
- Potential filesystem discovery, credential discovery, and lateral movement attempts.
- Risk increases if the container runs with elevated privileges, mounted secrets, or writable host paths.
- Could become a cluster-impacting issue when combined with weak Kubernetes isolation.

Remediation

- Do not use shell=True with user input.
- Pass command arguments as a list.
- Use strict allowlisting for hostnames or IP addresses.
- Consider removing OS command execution entirely and using a safe library call where possible.

```
subprocess.run(
    ["ping", "-c", "1", host],
    shell=False,
    capture_output=True,
    text=True
)
```

Validation

Re-test with command separators such as semicolon, ampersand, pipe, and backticks. The application should reject invalid host values or treat them as invalid input, not as executable shell syntax.

```
Normal ping test:
user@ping 127.0.0.1 (127.0.0.1) 56(80) bytes of data:
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.154 ms

--- 127.0.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 1ms
rtt min/avg/max/mdev = 0.154/0.154/0.154/0.000 ms

~/pre>
[command injection test with id:
user@ping 127.0.0.1 (127.0.0.1) 56(80) bytes of data:
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.063 ms

--- 127.0.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.063/0.063/0.063/0.000 ms
user@ping(jazz) ping[ping(jazz) group=1800(jazz),v(cdn),2l(cdn),27(audio),3l(
ftp),4d(plugin),118(lcd),999(docker)]

~/pre>
[command injection test with whoami:
user@ping 127.0.0.1 (127.0.0.1) 56(80) bytes of data:
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.060 ms

--- 127.0.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.060/0.060/0.060/0.000 ms
jazz

~/pre>
(.venv) jazz@ubuntu-vm:~/devsecops-lab$ curl "http://local
host:8080/ping?host=127.0.0.1;id;whoami;cat/etc/passwd;ls;/etc/passwd"
user@ping 127.0.0.1 (127.0.0.1) 56(80) bytes of data:
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.062 ms

--- 127.0.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.062/0.062/0.062/0.000 ms
command: ls;/etc/passwd

~/pre>(.venv) jazz@ubuntu-vm:~/devsecops-lab$
```

Evidence: Command injection proof captured in the private lab.

FND-003	Vulnerable Python dependency exposure	High	7.5	Mitigated by pipeline control
Affected Component	src/app/requirements.txt			
Attack Class	Software Composition Analysis finding			
Evidence	pip-audit identified known dependency vulnerabilities			
Business Impact	Known vulnerable libraries can expose the application to publicly documented exploit paths			

Description

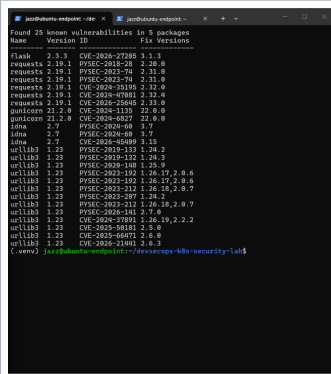
The application intentionally included an outdated dependency version to demonstrate SCA detection. Dependency vulnerabilities are especially risky because they can enter the build without direct changes to application code.

Remediation

- Upgrade vulnerable packages to secure supported versions.
- Pin dependencies through a lockfile or controlled requirements process.
- Run pip-audit in CI for every pull request.
- Fail builds when critical or high vulnerabilities are found unless a documented exception exists.
- Schedule periodic dependency review because new CVEs may appear after deployment.

Validation

The CI/CD pipeline includes dependency scanning and stores reports as artifacts. Remediation is validated when pip-audit no longer reports unresolved high-risk findings for production dependencies.



Evidence: pip-audit dependency scan output in CI/local testing.

FND-004	Container image vulnerability and hardening gap	High	7.2	Remediated
Affected Component	Docker image and Dockerfile			
Attack Class	Container image vulnerability and insecure runtime baseline			
Evidence	Trivy image scan and Docker hardening validation			
Business Impact	Attackers may exploit vulnerable packages or abuse overly permissive container runtime settings			

Description

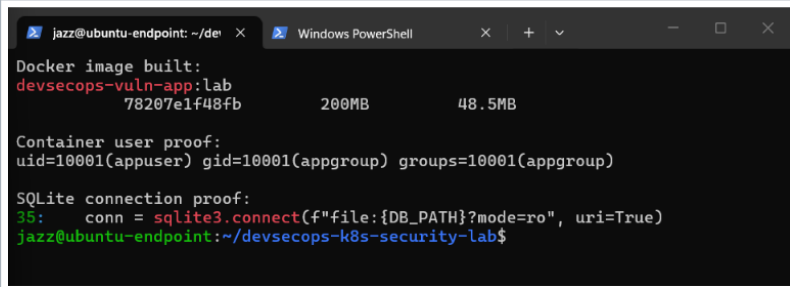
Container images may contain vulnerable base packages, application dependencies, unnecessary build tooling, or insecure runtime defaults. The lab addressed this by implementing a multi-stage Docker build, running as a non-root user, and scanning the image with Trivy.

Remediation

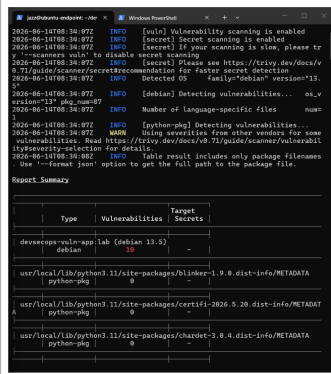
- Use minimal base images and rebuild regularly.
- Use multi-stage builds to keep build tools out of the runtime image.
- Create a fixed non-root user and run the application as that UID.
- Avoid package manager cache and unnecessary files.
- Run Trivy image scans locally and in CI/CD.
- Use severity gates for critical and high vulnerabilities.

Validation

The final image was rebuilt, executed as a non-root user, and scanned with Trivy. CI/CD image scan evidence was captured as part of the project artifacts.



Evidence: Docker image running as a non-root UID.



Evidence: Trivy image scan for the DevSecOps application image.

FND-005	Insecure Kubernetes pod security context	Critical	9.1	Remediated
Affected Component	Kubernetes Deployment manifest			
Attack Class	Weak workload isolation and privilege boundaries			
Evidence	SecurityContext review and hardened manifest validation			
Business Impact	Containers with weak isolation can become stepping stones toward node or cluster compromise			

Description

The Kubernetes workload required strong pod-level and container-level hardening controls. Without these controls, the workload could run with unnecessary privileges, write to the root filesystem, inherit Linux capabilities, or access mounted service account tokens.

Remediation

- Set runAsNonRoot: true and use a fixed non-root UID/GID.
- Set allowPrivilegeEscalation: false.
- Set readOnlyRootFilesystem: true and provide a limited /tmp emptyDir when needed.
- Drop all Linux capabilities.
- Use seccompProfile: RuntimeDefault.
- Disable automountServiceAccountToken unless the pod needs Kubernetes API access.
- Set CPU and memory requests/limits.

Validation

Validation confirmed the pod security context, container security context, and read-only filesystem behavior. Attempts to write into protected application paths failed as expected.

```

Containers:
  app:
    Container ID: containerd://c9b18e4d8808cc3a2b5d0e33a37719831be9cb
    Image ID: docker.io/library/important-2025-06-14gha256:78297e1f48fbc
    Port: 5000/TCP (http)
    Host Port: 5000/TCP (http)
    State: Running
    Started: Sun, 14 Jun 2026 15:41:29 +0000
    Restart Count: 0
    Limits:
      cpu: 200m
      memory: 250Mi
    Requests:
      cpu: 50m
      memory: 60Mi
    Liveness: http-get http://:5000/health delay=15s timeout=2s period=10s
    Readiness: http-get http://:5000/health delay=5s timeout=2s period=10s
    Environment:
      - name:
        value: /tmp from tap (rw)
    Mounts:
      - name: demo-app-cf45bf66-km488
        mountPath: /app
        readOnly: true
    Conditions:
      - type: PodReady
        status: True

```

Evidence: Hardened pod security context visible in Kubernetes object details.

```

Pod name: demo-app-cf45bf66-km488

Test writing to /app. Expected: Read-only file system
touch: cannot touch '/app/test.txt': Read-only file system
command terminated with exit code 1

Test writing to /tmp. Expected: tmp-write-ok
tmp-write-ok
jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$

```

Evidence: Read-only root filesystem validation.

FND-006	Privileged pod with hostPath container escape pattern	Critical	9.3	Remediated
Affected Component	runtime-lab privileged pod			
Attack Class	Container escape pattern using privileged container and hostPath mount			
Evidence	Privileged pod accessed node filesystem through /host			
Business Impact	Potential node compromise and cluster pivot if allowed in production			

Description

A privileged pod with a hostPath mount was deployed to demonstrate a high-risk Kubernetes misconfiguration. The container mounted the node root filesystem at /host and then used chroot to interact with that filesystem. In a Kind lab this demonstrates the attack pattern safely against the Kind node container.

Impact

- Access to host or node filesystem through mounted paths.
- Potential reading of sensitive node configuration.
- Possible tampering with host files if the mount is writable.
- A realistic path toward escaping normal container isolation.

Remediation

- Block privileged containers using Pod Security restricted enforcement.
- Avoid hostPath mounts unless absolutely required and tightly controlled.
- Use admission control to reject dangerous pod specifications.
- Monitor runtime behavior using Falco and alert on sensitive file access or privileged activity.

Validation

After Pod Security restricted enforcement was applied, Kubernetes rejected the same privileged pod specification. Falco detection evidence was also captured during the simulated attack.

```
jazz@ubuntu-endpoint: ~/dev  jazz@ubuntu-endpoint: ~/dev  + - - - - -
Container escape simulation using privileged pod + hostPath

Inside attacker container:
uid=0(root) gid=0(root) groups=0(root),1(bin),2(demon),3(sys),4(adm),6(disk),16(wheel),116(floppy),98(cialout),26(tape),27(video)

Can see mounted node filesystem:
LICENSES
bin
boot
dev
etc
home
kind
lib
lib64
media

Node hostname file:
devsecops-lab-control-plane

Chroot into mounted node filesystem:
Now running inside mounted node root:
root
attacker-privileged
jazz@ubuntu-endpoint: ~/devsecops-k8s-security-lab$
```

Evidence: Privileged pod accessing mounted node filesystem through /host.

```
jazz@ubuntu-endpoint: ~/dev  jazz@ubuntu-endpoint: ~/dev  + - - - - -
Falco detected privileged pod activity:
Defaulted container "falco" out of: falco, falcoctl-artifact-follow, falcoctl-artifact-install (init)
16:01:27.530306375: Warning Sensitive file opened for reading by non-trusted
program | file=/etc/shadow gparent=<NA> gpparent=<NA> gggparent=<NA> evt_type=open user=root user_uid=0 user_loginuid=-1 process=cat proc_exepath=/bin/busybox parent=sh command=cat /etc/shadow terminal=0 container_id=f9383142676b container_name=attacker container_image_repository=docker.io/library/alpine container_image_tag=3.20 k8s_pod_name=attacker-privileged k8s_ns_name=runtime-lab
jazz@ubuntu-endpoint: ~/devsecops-k8s-security-lab$
```

Evidence: Falco detection from privileged pod activity.

```
jazz@ubuntu-endpoint: ~/dev  jazz@ubuntu-endpoint: ~/dev  + - - - - -
Testing same privileged pod after restricted Pod Security:
Error from server (Forbidden): error when creating "redteam/runtime/privileged-hostpath-pod.yaml": pods "attacker-privileged" is forbidden: violates Pod Security "restricted:latest": privileged (container "attacker" must not set securityContext.privileged=true), allowPrivilegeEscalation != false (container "attacker" must set securityContext.allowPrivilegeEscalation=false), unrestricted capabilities (container "attacker" must set securityContext.capabilities.drop=["ALL"]), restricted volume types (volume "host-root" uses restricted volume type "hostPath"), runAsNonRoot != true (pod or container "attacker" must set securityContext.runAsNonRoot=true), seccompProfile (pod or container "attacker" must set securityContext.seccompProfile.type to "RuntimeDefault" or "Localhost")

Pods in runtime-lab:
NAME      READY   STATUS    RESTARTS   AGE
falco-test 0/1     Completed 0           59m
jazz@ubuntu-endpoint: ~/devsecops-k8s-security-lab$
```

Evidence: Pod Security restricted enforcement blocking the privileged pod.

FND-007	Overprivileged ServiceAccount with cluster-admin access	Critical	9.0	Remediated
Affected Component	Kubernetes RBAC			
Attack Class	Privilege escalation through ClusterRoleBinding			
Evidence	ServiceAccount could access secrets and cluster-wide resources			
Business Impact	A compromised token could lead to full cluster takeover			

Description

The lab intentionally created a ServiceAccount and bound it to the built-in cluster-admin ClusterRole. This granted full administrative power across the Kubernetes cluster. The issue was validated using kubectl auth can-i checks.

Impact

- Read Kubernetes secrets across namespaces.
- Create privileged pods or malicious workloads.
- Modify or delete workloads and security controls.
- Create additional ClusterRoleBindings and maintain persistence.
- Potential full cluster compromise.

Remediation

- Delete unnecessary ClusterRoleBindings.
- Replace cluster-wide access with namespace-scoped Roles and RoleBindings.
- Grant only specific verbs and resources required by the workload.
- Continuously audit subjects bound to cluster-admin.
- Disable automatic token mounting for workloads that do not need Kubernetes API access.

Validation

After remediation, the ServiceAccount could list ConfigMaps in its namespace but could no longer access secrets or create ClusterRoleBindings.

```

jazz@ubuntu-endpoint: ~/dev
jazz@ubuntu-endpoint: ~/dev
=== BEFORE FIX: overprivileged-sa permissions ===
Can do everything cluster-wide?
yes
Can get secrets cluster-wide?
yes
Can create clusterrolebindings?
Warning: resource 'clusterrolebindings' is not namespace scoped in group 'rbac.authorization.k8s.io'
yes
jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$

```

Evidence: Overprivileged ServiceAccount could access secrets before remediation.

```

Windows PowerShell
jazz@ubuntu-endpoint: ~/dev
=== AFTER FIX: least privilege permissions ===
Can list configmaps in rbac-lab?
yes
Can get secrets in rbac-lab?
no
Can get secrets cluster-wide?
no
Can create clusterrolebindings?
Warning: resource 'clusterrolebindings' is not namespace scoped in group 'rbac.authorization.k8s.io'
no
jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$

```

Evidence: Least-privilege RBAC fix prevents secret access.

FND-008	Missing default-deny network segmentation	High	7.4	Remediated
Affected Component	Kubernetes namespace networking			
Attack Class	Lateral movement risk through unrestricted pod-to-pod traffic			
Evidence	NetworkPolicy allowed trusted client and blocked untrusted client			
Business Impact	Compromised pods may move laterally if network paths are unrestricted			

Description

Kubernetes environments often allow pod-to-pod communication by default unless NetworkPolicy enforcement is configured. The lab used Calico to enforce a default-deny model and then explicitly allowed trusted ingress and required DNS egress.

Remediation

- Install a NetworkPolicy-enforcing CNI such as Calico.
- Apply default-deny ingress and egress policies.
- Permit only required application traffic.

- Allow DNS egress explicitly.
- Test allowed and blocked flows with dedicated curl pods.

Validation

An untrusted pod was blocked from reaching the service, while an approved pod with the correct labels was allowed. This validated both prevention and intended application connectivity.

```

jazz@ubuntu-endpoint: ~/devsecops-k8s-security-lab$ kubectl get networkpolicy -n devsecops
NAME                                POD-SELECTOR  AGE
allow-approved-client-egress-to-app  access=allowed 1s
allow-demo-app-ingress               app=demo-app   1s
allow-dns-egress                     <none>         1s
default-deny-all                    <none>         1s
jazz@ubuntu-endpoint: ~/devsecops-k8s-security-lab$

```

Evidence: NetworkPolicies created in the devsecops namespace.

```

jazz@ubuntu-endpoint: ~/devsecops-k8s-security-lab$ curl -s http://demo-app:8080/
Blocked untrusted pod test:
curl: (28) Connection timed out after 5005 milliseconds
command terminated with exit code 28

jazz@ubuntu-endpoint: ~/devsecops-k8s-security-lab$ kubectl get pods -n devsecops
NAME                                READY  STATUS   RESTARTS  AGE    LABELS
blocked-curl                        1/1    Running  0          20s    test=blocked
demo-app-cf45bf66-km488             1/1    Running  0          29m    app=demo-app,po
d-template-hash=cf45bf66
jazz@ubuntu-endpoint: ~/devsecops-k8s-security-lab$

```

Evidence: Untrusted pod blocked by NetworkPolicy.

```

jazz@ubuntu-endpoint: ~/devsecops-k8s-security-lab$ curl -s http://demo-app:8080/
Allowed trusted pod test:
{"service": "vulnerable-flask-app", "status": "ok"}

jazz@ubuntu-endpoint: ~/devsecops-k8s-security-lab$ kubectl get pods -n devsecops
NAME                                READY  STATUS   RESTARTS  AGE    LABELS
allowed-curl                        1/1    Running  0          23s    access=allowe
d
blocked-curl                        1/1    Running  0          3m27s  test=blocked
demo-app-cf45bf66-km488             1/1    Running  0          32m    app=demo-app,po
d-template-hash=cf45bf66
jazz@ubuntu-endpoint: ~/devsecops-k8s-security-lab$

```

Evidence: Trusted pod allowed by NetworkPolicy.

FND-009	Kubernetes configuration drift without GitOps control	Medium	6.0	Remediated
Affected Component	Kubernetes deployment lifecycle			
Attack Class	Configuration drift and unauthorized operational change			
Evidence	Manual replica drift was self-healed by ArgoCD			
Business Impact	Untracked changes can bypass review, approvals, and security baselines			

Description

Manual kubectl changes can cause live cluster state to drift from the intended state stored in Git. The lab demonstrated this by scaling the deployment manually and allowing ArgoCD to detect and self-heal the drift.

Remediation

- Use ArgoCD or another GitOps controller as the deployment authority.
- Enable automated sync for controlled lab environments.
- Enable selfHeal to correct drift.
- Enable prune to remove resources deleted from Git.
- Treat Git as the source of truth and restrict manual production changes.

Validation

The deployment was manually scaled away from the Git-defined replica count. ArgoCD restored the replica count to match Git, proving drift correction.

```

jazz@ubuntu-endpoint: ~ - /de x jazz@ubuntu-endpoint: ~
Final ArgoCD application status:
NAME      SYNC STATUS  HEALTH STATUS
demo-app  Synced      Healthy

Sync: Synced | Health: Healthy

Pods:
NAME      NODE      READY  STATUS  RESTARTS  AGE  IP
allowed-curl 0/1      Completed 0      17h     192.168.1
49.8 devsecops-lab-control-plane <none> 0      18h     192.168.1
blocked-curl 0/1      Completed 0      18h     192.168.1
49.7 devsecops-lab-control-plane <none> 0      57s     192.168.1
demo-app-69f8fd4fd-st7p7 1/1      Running 0      18h     192.168.1
49.16 devsecops-lab-control-plane <none> 0      18h     192.168.1
demo-app-cf45bf66-kw488 0/1      Terminating 0      18h     192.168.1
49.5 devsecops-lab-control-plane <none> 0      18h     192.168.1
jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$

```

Evidence: ArgoCD application synchronized and healthy.

```

jazz@ubuntu-enc x jazz@ubuntu-enc x jazz@ubuntu-enc x + - - -
Waiting for ArgoCD self-heal back to Git-defined replicas...
Check 1/60
NAME      READY  UP-TO-DATE  AVAILABLE  AGE
demo-app  1/1    1           1           19h

NAME      READY  STATUS  RESTARTS  AGE
demo-app-69f8fd4fd-st7p7 1/1      Running 0          32m
demo-app-cf45bf66-kw488 0/1      Terminating 0          19h

NAME      SYNC STATUS  HEALTH STATUS
demo-app  Synced      Healthy

Final self-healed state:
NAME      READY  UP-TO-DATE  AVAILABLE  AGE
demo-app  1/1    1           1           19h

Git desired state:
%: replicas: 1

ArgoCD status:
NAME      SYNC STATUS  HEALTH STATUS
demo-app  Synced      Healthy
jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$

```

Evidence: ArgoCD self-healed manual replica drift.

FND-010	Unsigned image and missing SBOM/provenance evidence	High	7.2	Remediated
Affected Component	Container supply chain			
Attack Class	Lack of artifact integrity and provenance evidence			
Evidence	Cosign signature, Syft SBOM, SBOM attestation, SLSA-style provenance			
Business Impact	Weak artifact trust and poor auditability of what was deployed			

Description

Before supply-chain controls were added, the container image lacked cryptographic proof of origin and a formal software bill of materials. This reduces confidence that the deployed image matches the intended build and makes audit or incident response more difficult.

Remediation

- Push the image to GHCR using an authenticated workflow.
- Sign the image with Cosign.
- Generate SPDX and CycloneDX SBOMs using Syft.
- Attach and attest the SBOM to the image.
- Generate SLSA-style provenance evidence for the private lab build.
- Verify signatures and attestations as part of release validation.

Validation

Cosign successfully verified the image signature and attestations. Syft generated SBOM artifacts, and provenance evidence was stored with the project documentation.

```

jazz@ubuntu-enc x Windows Powershell x Windows Powershell x + - - -
Current image digest:
ghcr.io/jazz08081/enterprise-devsecops@sha256:14776e0fe1b95a6aab7a76d5715613
d359a9e92be22716b44c1f7514bc728764

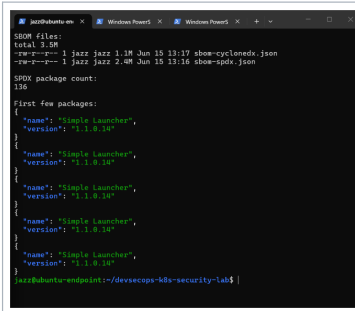
Signing current digest:
Enter password for private key:
Signing artifact... \ Pushing signature to: ghcr.io/jazz08081/enterprise-dev
secops

Verifying Cosign image signature:
Verification for ghcr.io/jazz08081/enterprise-devsecops@sha256:14776e0fe1b95
a6aab7a76d5715613d359a9e92be22716b44c1f7514bc728764 -
The following checks were performed on each of these signatures:
- The cosign claims were validated
- Existence of the claims in the transparency log was verified offline
- The signatures were verified against the specified public key

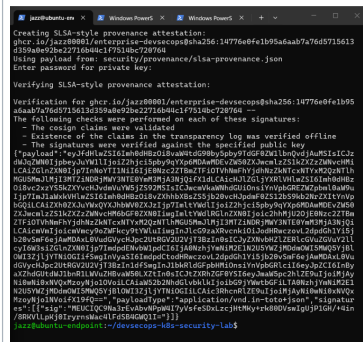
[{"critical":{"identity":{"docker-reference":"ghcr.io/jazz08081/enterprise-d
evsecops@sha256:14776e0fe1b95a6aab7a76d5715613d359a9e92be22716b44c1f7514bc72
8764"},"image":{"docker-manifest-digest":"sha256:14776e0fe1b95a6aab7a76d5715
613d359a9e92be22716b44c1f7514bc728764"},"type":"https://sigstore.dev/cosign/
sign/v1"},"optional":{"}}]
jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$

```

Evidence: Cosign signature verification succeeded.



Evidence: Syft SBOM generation.



Evidence: SLSA-style provenance attestation verified.

FND-011	Missing admission-control policy for supply-chain metadata	Medium	6.4	Remediated
Affected Component	Kubernetes admission control			
Attack Class	Governance and auditability gap			
Evidence	OPA Gatekeeper ConstraintTemplate and Constraint			
Business Impact	Workloads can enter the cluster without ownership or supply-chain labels			

Description

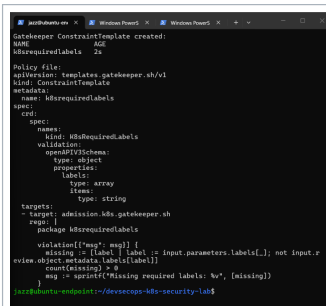
Without admission control, Kubernetes Deployments could be submitted without metadata indicating ownership, signing status, or SBOM status. This makes security review, inventory management, and audit response weaker.

Remediation

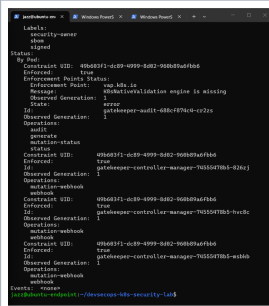
- Install OPA Gatekeeper.
- Create a reusable ConstraintTemplate for required labels.
- Create a Constraint requiring security-owner, sbom, and signed labels on Deployments.
- Document the policy-as-code control in the repository.
- Validate the rule with compliant and non-compliant deployment examples.

Validation

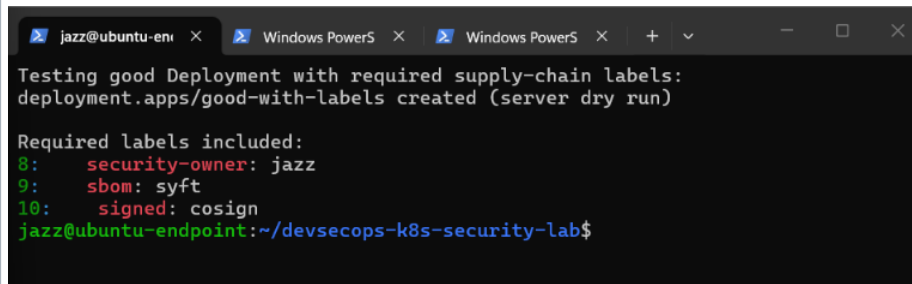
Gatekeeper was installed and the required-labels policy was created. A compliant deployment containing the required labels was allowed by the server-side dry run.



Evidence: Gatekeeper required-label ConstraintTemplate created.



Evidence: Gatekeeper required-label Constraint created.



Evidence: Compliant deployment allowed by policy.

FND-012	Runtime security and observability visibility gaps		Medium	6.2	Remediated
Affected Component	Runtime detection and monitoring stack				
Attack Class	Detection and visibility gap				
Evidence	Falco, Falcosidekick, Prometheus, Grafana dashboards and alert rules				
Business Impact	Security events may be missed or remain unactioned without detection and dashboards				

Description

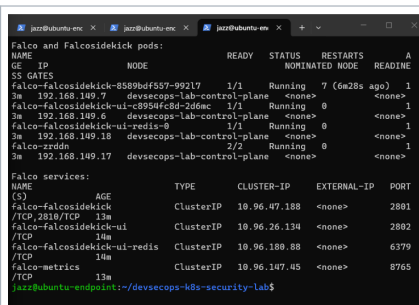
Preventive controls reduce risk, but runtime detection is needed when suspicious behavior occurs after deployment. The lab integrated Falco for runtime security events, Falcosecurity for event forwarding/metrics, Prometheus for collection, and Grafana for dashboards.

Remediation

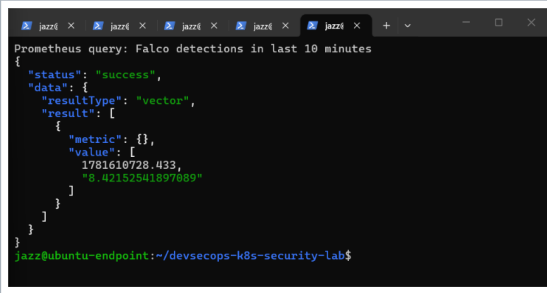
- Install Falco in the Kubernetes cluster.
- Deploy Falcosidekick and expose Prometheus metrics.
- Install kube-prometheus-stack with Prometheus and Grafana.
- Create ServiceMonitors so Prometheus scrapes Falco/Falcosidekick metrics.
- Build a custom Grafana dashboard for runtime security posture.
- Create Prometheus alert rules for Falco detections and event drops.

Validation

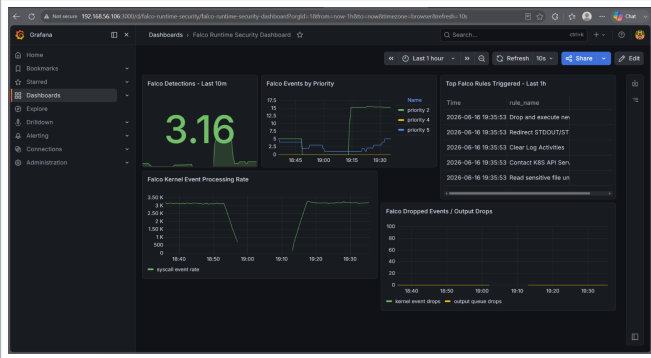
Falco events were generated safely in the lab, Prometheus queried Falco rule matches, and Grafana dashboards displayed runtime security metrics.



Evidence: Falco and Falcosidekick running in the cluster.



Evidence: Prometheus query showing Falco rule matches.



Evidence: Custom Grafana runtime security dashboard.

5. Consolidated Remediation Roadmap

The following roadmap groups the remediation work by operational priority. This format is suitable for presenting to engineering, security, or hiring panels because it shows how isolated findings become a coordinated security improvement plan.

Priority	Action
Immediate	Block command injection and SQL injection patterns; remove shell=True; parameterize queries; validate input at the route boundary.
Immediate	Remove cluster-admin bindings from application ServiceAccounts; enforce least-privilege RoleBindings and verify with kubectl auth can-i.
Immediate	Enforce Pod Security restricted labels and block privileged pods, hostPath mounts, privilege escalation, and root execution.
Short Term	Run Bandit, pip-audit, Trivy image scans, and Trivy config scans in CI/CD with defined severity thresholds.
Short Term	Apply Kubernetes hardening controls: non-root UID, read-only filesystem, dropped capabilities, RuntimeDefault seccomp, resource limits, probes.
Short Term	Deploy Calico NetworkPolicies using default-deny ingress/egress and explicit service allow rules.
Medium Term	Implement ArgoCD GitOps with self-healing and pruning to control drift and align cluster state to Git.
Medium Term	Sign images with Cosign, generate Syft SBOMs, attach/attest SBOMs, and verify provenance during release validation.
Medium Term	Enforce supply-chain metadata through Gatekeeper admission policies and include policy tests in the repo.
Ongoing	Monitor Falco detections in Prometheus/Grafana, review alerts, audit RBAC bindings, and periodically rescan dependencies and images.

6. Retest and Validation Matrix

Retesting confirms whether remediation is effective. Each validation step maps directly to a finding and uses objective command output, scan reports, or screenshots as evidence.

Finding	Validation Criteria
FND-001	SQL injection payload no longer returns unauthorized records; parameterized query verified in code review.
FND-002	Command separators are rejected or treated as invalid input; shell=True removed from command execution.
FND-003	pip-audit report shows no unresolved high-risk dependency issues for production packages.
FND-004	Trivy image scan reviewed; image runs as non-root UID; Dockerfile uses multi-stage build.
FND-005	kubectl output confirms runAsNonRoot, readOnlyRootFilesystem, dropped capabilities, and seccomp RuntimeDefault.
FND-006	Privileged pod is rejected by Pod Security restricted enforcement; Falco alerts captured during simulation.
FND-007	kubectl auth can-i confirms ServiceAccount cannot read secrets or create ClusterRoleBindings.
FND-008	Untrusted pod connection times out while trusted pod successfully reaches the service.
FND-009	ArgoCD restores manual replica drift and reports Synced/Healthy state.
FND-010	Cosign verify and verify-attestation commands succeed; SBOM exists in SPDX/CycloneDX formats.
FND-011	Gatekeeper policy exists and compliant workload passes server-side dry run.
FND-012	Prometheus has Falco targets up and Grafana displays runtime security dashboard panels.

7. Control Mapping

This matrix maps the remediation work to common security engineering control domains. It is not a formal compliance audit, but it helps communicate the security value of the lab in enterprise language.

Control Domain	Implemented Evidence
Secure SDLC	Bandit SAST, pip-audit SCA, Trivy filesystem/image/config scans, GitHub Actions workflow artifacts
Container Security	Multi-stage Dockerfile, non-root runtime, healthcheck, image scanning, minimal runtime surface
Kubernetes Hardening	SecurityContext, Pod Security restricted, RBAC least privilege, NetworkPolicy default deny, resource limits
Supply Chain Security	Cosign signing, Syft SBOM, SBOM attestation, provenance attestation, GHCR publishing
Policy-as-Code	OPA Gatekeeper ConstraintTemplates and Constraints enforcing required deployment metadata
Runtime Detection	Falco rules and alerts for sensitive file access and privileged behavior
Observability	Prometheus targets, Grafana dashboards, Falco metrics, PrometheusRule alerting
Change Control	ArgoCD GitOps automated sync, self-healing, pruning, and drift correction

8. Final Security Posture

The final lab posture demonstrates a layered DevSecOps model. The application layer is tested for code-level vulnerabilities, the container layer is hardened and scanned, Kubernetes resources are restricted through least privilege and policy, the supply chain is signed and documented, runtime events are detected, and operational visibility is provided through dashboards.

**Professional
Assessment Note**

This report intentionally separates finding, impact, remediation, and validation. That structure is important because recruiters and security teams want to see not only that tools were run, but that the results were interpreted, prioritized, fixed, and proven through evidence.

The project was completed in a private lab environment by Jagriti Banerjee and is suitable as a portfolio artifact for DevSecOps, cloud security, Kubernetes security, application security, and security engineering roles.

Appendix A - Evidence Reference Index

Evidence File	Purpose
035-sql-injection-proof.png	SQL injection proof for FND-001
036-command-injection-proof.png	Command injection proof for FND-002
042-pip-audit-dependency-findings.png	Dependency scan evidence for FND-003
048-trivy-image-scan-devsecops-app.png	Container image scan evidence for FND-004
079-pod-security-context-visible.png	Kubernetes security context evidence for FND-005
135-container-escape-chroot-proof.png	Privileged hostPath pod proof for FND-006
136-falco-detected-privileged-pod-activity.png	Falco detection evidence for FND-006
140-rbac-overprivileged-sa-can-get-secrets.png	RBAC privilege escalation evidence for FND-007
143-rbac-fixed-cannot-get-secrets.png	RBAC remediation evidence for FND-007
084-networkpolicy-blocked-untrusted-pod.png	NetworkPolicy blocked flow evidence for FND-008
103-argocd-self-healed-back-to-git-state.png	GitOps drift correction evidence for FND-009
115-cosign-signature-verified.png	Cosign verification evidence for FND-010
121-slsa-provenance-attestation-verified.png	Provenance attestation evidence for FND-010
123-gatekeeper-constrainttemplate-created.png	Gatekeeper policy template evidence for FND-011
158-custom-falco-security-dashboard-imported.png	Grafana security dashboard evidence for FND-012

Appendix B - Recommended Resume Summary

Built an end-to-end Enterprise DevSecOps Security Lab using Docker, GitHub Actions, Kind Kubernetes, ArgoCD, Trivy, Cosign, Syft, OPA Gatekeeper, Falco, Prometheus, and Grafana. Documented application security flaws, Kubernetes misconfigurations, RBAC escalation, container escape patterns, supply-chain gaps, runtime detection, remediation steps, and validation evidence in a professional findings-and-remediation format.